Course Outline: Telemetry Architecture

Title: Telemetry Architecture **Skill:** 42 — Identificar oportunidades de recolección de datos en escenarios reales que permitan optimizar compañías **Learnpack:** + Arquitectura y diseño de mecanismos de telemetría **File:** s42-w14d41-arquitectura-y-diseno-de-mecanismos-de-t **Prerequisite:** Introduction to Data Pipelines (Skill 39) **Target Audience:** Beginner developers building applications that need to collect and process user or system data **Total Lessons:** 10 **Format:** Conceptual (0% hands-on)

Course Description

Every modern application generates data — clicks, errors, timings, events — but collecting and processing that data well requires deliberate architectural decisions. This course covers the fundamentals of telemetry architecture: where to process data, how to transport it, how often to send it, and whether to handle it as a stream of individual events or as scheduled batches. These decisions determine whether a telemetry system is reliable, efficient, and scalable — or a source of bugs and bottlenecks.

Course Philosophy

This course emphasizes:

- Architecture as a set of trade-offs, not a checklist of rules
- Each decision (where, how, when, what model) understood in terms of its cost and benefit
- Practical framing: real systems make these choices every day, and the wrong choice has real consequences

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Telemetry Architecture	4
02 - Processing Models	3
03 - Assessment	1
04 - Outro	1
Total	**10**

00.0 Welcome to Telemetry Architecture 🕮

Learning Objectives:

- Understand why telemetry has become a core concern in modern software development
- Preview the architectural decisions this course will equip you to make

Content Outline:

- The shift: applications used to run without observability; now data collection is expected
- What telemetry is: the automatic collection of measurements and data from a running system
- The four decisions this course covers: where to process, how to transport, how often to send, and which processing model to use
- What you will be able to design by the end of this course

Transitions: This lesson frames the problem. Section 01 dives into the first two decisions: where data gets processed and how it travels.

Key Principles:

- Telemetry is not optional in production systems — it is how you know what is actually happening
- Every architectural decision in telemetry involves a trade-off: usually between cost/complexity and richness/reliability

Examples:

- Spotify collects every skip, replay, and failed search — not to spy on users, but to feed recommendation algorithms that keep people listening
- A web app with no telemetry: when users report a bug, you have no idea what happened or how often it occurs

---

01.0 Telemetry Architecture 🕮

Learning Objectives:

- Describe the three core architectural decisions that define a telemetry collection system
- Explain how client-side, server-side, and transport decisions interact
- Identify which decisions need to be made before building any collection mechanism

Content Outline:

- What a telemetry architecture consists of: collection point, transport layer, processing layer
- The three decisions covered in this section: WHERE to process, HOW to transport, HOW OFTEN to send
- Why these decisions must be made upfront: changing them later is expensive
- Overview of what 01.1, 01.2, and 01.3 will each cover

Transitions: This overview frames the section. Lesson 01.1 starts with the most fundamental decision: where does processing happen?

Key Principles:

- Architecture is decided before code — these are design questions, not implementation questions
- The three decisions are interdependent: where you process affects how you transport, which affects how often you can send

Examples:

- A mobile app deciding whether to analyze crash data on the device or send raw logs to the server — the answer changes the payload size, the battery impact, and the privacy surface
- A checkout flow: every payment attempt is a write to the database AND a telemetry event — the telemetry path needs its own architecture

---

01.1 Client vs Server Processing 🕮

Learning Objectives:

- Distinguish between processing telemetry on the client and processing it on the server
- Evaluate when each approach is appropriate based on the use case
- Identify the trade-offs: bandwidth, privacy, latency, and reliability

Content Outline:

- What "processing on the client" means: filtering, aggregating, or enriching data before sending it
- What "processing on the server" means: sending raw events and letting the backend handle everything
- Factors that favor client-side processing: bandwidth constraints, privacy requirements, reducing server load, faster local feedback
- Factors that favor server-side processing: more compute, centralized logic, richer cross-session analysis, easier updates
- Hybrid approaches: process lightly on the client, enrich on the server
- Other processing locations: edge servers, other backend services

Transitions: Once you know where processing happens, the next question is how data travels from the collection point to its destination. Lesson 01.2 covers transport.

Key Principles:

- Client-side processing reduces what you send, but limits what you can analyze later
- Server-side processing preserves raw data, but costs bandwidth and processing power
- The right answer depends on the constraints of the device and the goals of the analysis

Examples:

- Mobile analytics SDK (e.g. Firebase): samples, filters, and batches events on the device before uploading — because mobile data plans and battery matter
- A web app error tracker: sends the full stack trace raw to the server — because the client doesn't have enough context to enrich it meaningfully
- Anti-pattern: sending every raw mouse movement to the server — the volume is enormous, the signal is low, and the bandwidth cost is unjustifiable

---

01.2 Data Transport 🕮

Learning Objectives:

- Explain what "viability of sending data" means in a telemetry context
- Identify the key factors that determine whether a telemetry payload can be sent reliably
- Choose an appropriate serialization format and transport mechanism for a given scenario

Content Outline:

- What transport viability means: can the data actually get from the collection point to the processing point given real-world constraints?
- Payload size: how much data can reasonably be sent without impacting the user experience or exceeding network limits
- Serialization: converting data structures into a format suitable for transport
  - JSON: human-readable, widely supported, larger payloads
  - Binary formats (e.g. Protocol Buffers, MessagePack): compact, fast, less readable
  - The trade-off: readability vs. efficiency
- Transport mechanisms: HTTP (simple, universal), WebSockets (persistent, real-time), sendBeacon (fire-and-forget, reliable on page unload)
- Reliability concerns: what happens when the network is unavailable — retry queues, local buffering
- When transport fails: the event is lost vs. stored locally and retried

Transitions: You now know where to process and how to send. Lesson 01.3 covers the remaining architectural decision: how often to send.

Key Principles:

- Serialization format is a contract — changing it later requires coordinating both sides simultaneously
- Binary formats are faster and smaller but add tooling complexity — choose them when payload size is a real constraint
- Always design for failure: network calls can and will fail; your telemetry system needs a plan

Examples:

- A browser sending events via `navigator.sendBeacon` on `pagehide`: the request completes even if the page is closing, making it reliable for exit events
- A mobile app storing events in a local SQLite queue when offline, then flushing when connectivity returns
- Anti-pattern: choosing a binary format for a low-volume internal tool — the added complexity of debugging binary payloads outweighs the tiny size savings

---

01.3 Throttling 🕮

Learning Objectives:

- Define throttling in the context of telemetry

- Explain why unconstrained event emission is a system risk
- Apply throttling strategies to control data volume without losing meaningful signal

Content Outline:

- What throttling is: limiting the rate at which events are emitted or sent
- Why it matters: a system without throttling can flood its own backend, degrade user experience, or generate bills it cannot pay
- Common throttling strategies:
  - Rate limiting: maximum N events per second/minute
  - Sampling: send only X% of events, chosen randomly or by rule
  - Debouncing: collapse many rapid events into one (e.g. keystrokes, scroll events)
  - Deduplication: drop identical events within a time window
- The signal preservation problem: how do you throttle without losing the events that matter most?
- Priority-based throttling: always send errors, sample informational events

Transitions: You have now made all three collection-layer decisions: where to process, how to transport, and how often to send. Section 02 moves to the processing layer — what happens to the data once it arrives.

Key Principles:

- Throttling is not optional at scale — without it, telemetry becomes a DDoS attack on your own infrastructure
- Sampling at 10% of events still gives you statistically valid signal for most analyses — you do not need 100% of events to understand user behavior
- Critical events (errors, payment failures, security events) should never be sampled — always send them

Examples:

- A scroll event fires 60 times per second as the user scrolls — without debouncing, a 10-minute session generates 36,000 events for a single interaction
- Cloudflare processes 50M+ HTTP requests per second and uses aggressive sampling to make telemetry tractable — they do not record every packet
- Anti-pattern: sampling errors at 10% — you lose 90% of your most valuable signal and cannot reliably reproduce reported bugs

---

02.0 Processing Models 🕮

Learning Objectives:

- Describe the two fundamental processing models for telemetry data
- Explain how the choice of processing model follows from the use case
- Preview the specific characteristics of stream and batch processing

Content Outline:

- Once data arrives at the processing layer, the system must decide how to handle it
- The two models: stream processing (handle each event as it arrives) and batch processing (accumulate events and handle them together)
- This is not a binary choice — most real systems use both for different data types
- How the processing model connects back to transport: streaming data flows continuously, batch data is collected then flushed
- Overview of what 02.1 and 02.2 will cover

Transitions: This overview frames the choice. Lesson 02.1 examines stream processing in depth; lesson 02.2 covers batch processing.

Key Principles:

- Processing model determines latency: streams give you near-real-time results, batches give you periodic results
- Processing model determines cost: streams require always-on infrastructure, batches can run on a schedule and use fewer resources

Examples:

- Fraud detection: must use stream processing — a fraudulent transaction identified 30 minutes later (batch) is useless
- Monthly billing report: can use batch processing — the report is generated once a month, so real-time processing adds cost with no benefit

---

02.1 Stream Processing 🕮

Learning Objectives:

- Define stream processing and explain its core characteristic: atomic event handling
- Identify scenarios where stream processing is the appropriate model
- Describe the infrastructure requirements and trade-offs of stream processing

Content Outline:

- What stream processing is: each event is processed individually, as soon as it arrives
- "Atomic activity": the processing of one event is independent and self-contained — it does not wait for other events to arrive
- The latency profile: near-real-time; results are available within seconds or milliseconds of the event occurring
- Use cases: fraud detection, live dashboards, alerting, real-time recommendations, session tracking
- Infrastructure: requires a persistent, always-on processing pipeline (e.g. Kafka + a consumer, a WebSocket listener)
- The cost: stream infrastructure runs continuously — you pay even when no events are flowing
- Failure modes: if the processor goes down, events can be lost or delayed unless the queue persists them

Transitions: Stream processing excels when you need immediate results. Lesson 02.2 covers the alternative: batch processing, which trades latency for efficiency.

Key Principles:

- Stream processing is the right model when the value of an insight decays rapidly — fraud, errors, live metrics
- Atomic processing means each event must carry enough context to be processed in isolation — you cannot look back at previous events
- Always-on infrastructure is the main cost driver; do not use stream processing for data where a 1-hour lag is acceptable

Examples:

- A security system that detects a login from a new country and immediately triggers a verification email — the response must happen in seconds, not hours
- Live sports score updates: each score event is processed immediately and pushed to all connected clients
- Anti-pattern: using a streaming pipeline for a weekly cohort analysis report — the always-on infrastructure cost is wasted, and a batch job would do the same work at a fraction of the cost

---

02.2 Batch Processing 🕮

Learning Objectives:

- Define batch processing and explain its core characteristic: accumulated chunk handling
- Identify scenarios where batch processing is the appropriate model
- Describe the efficiency advantages and latency trade-offs of batch processing

Content Outline:

- What batch processing is: events are collected over a period of time, then processed together as a group
- "Big chunks": the processing unit is a set of events, not a single event — the processor sees many events at once
- The latency profile: periodic; results are available after the batch runs (minutes, hours, or days after events occur)
- Use cases: daily reports, weekly summaries, model training, billing calculations, data warehouse loads
- The efficiency advantage: processing a million events together is far cheaper than processing them one at a time — database queries, file I/O, and network calls can be shared
- Scheduling: batch jobs run on a schedule (cron) or when a threshold is reached (e.g. accumulate 10,000 events then process)
- Failure modes: if a batch fails, the entire period's data must be reprocessed — idempotency is critical

Transitions: You now understand both processing models. The assessment in section 03 will test your ability to apply these architectural decisions to realistic scenarios.

Key Principles:

- Batch processing is the right model when data volume is high, latency tolerance is measured in hours or days, and efficiency matters
- Batch jobs must be idempotent: running the same batch twice must produce the same result, not double the output
- The trigger for a batch can be time-based (run at midnight) or volume-based (process when 10K events accumulate) — the choice depends on how predictable the volume is

Examples:

- Netflix's weekly content performance report: billions of viewing events are aggregated into a single report — stream processing would be absurd at this scale and latency
- A machine learning training pipeline: collect user interaction data for a week, clean it in a batch job, feed the cleaned data to the model trainer
- Anti-pattern: running a batch job every second to "approximate" stream processing — this gives you the worst of both models: the latency of batch and the always-on cost of streaming

---

03.0 Telemetry Architecture in Practice 🫣

Learning Objectives:

- Apply the architectural decisions covered in this course to realistic system design scenarios
- Distinguish between appropriate and inappropriate choices for each decision dimension
- Evaluate trade-offs between competing architectural options

Question Format: Scenario-based

Questions:

1. A ride-sharing app needs to detect when a driver is speeding in real time and alert the operations team immediately. Which processing model should handle this data, and why?

2. A startup is building its first analytics pipeline. Their backend engineer suggests sending raw click events as JSON over HTTP. Their mobile engineer suggests using a binary format. The app has 500 daily active users. Which approach is better for now, and what would change that recommendation?

3. A web application sends a telemetry event on every keystroke in a search box. During a 30-second search session, this generates over 400 events. What throttling strategy would best preserve signal while reducing volume?

4. A company runs a nightly job that aggregates the previous day's user sessions into a cohort analysis report. One morning the job fails halfway through. What property must the job have to safely re-run it without corrupting the report?

5. An e-commerce platform processes payment events. The team debates whether to process them client-side (in the browser) or server-side. What factors determine the right answer, and which approach is almost certainly correct for payment data?

6. A social media platform wants to calculate its monthly active user (MAU) count. Should it use stream or batch processing? Justify your answer with reference to latency requirements and cost.

7. A mobile app collects crash reports. The app is used in areas with unreliable connectivity. What transport mechanism and failure strategy should the telemetry system use to ensure crash reports are not lost?

8. A team is designing a telemetry system for a high-traffic API (10,000 requests/second). They want to log every request for debugging purposes. Without throttling, how many events per hour would be generated, and what throttling strategy would make this tractable while preserving error visibility?

Evaluation Criteria:

- Correctly identifies which processing model (stream vs batch) fits each scenario based on latency and cost
- Applies throttling strategies appropriate to the volume and signal type
- Distinguishes client-side from server-side processing based on the data sensitivity and constraints
- Considers failure modes in transport and batch processing decisions

Score Interpretation:

- 7-8 correct: Ready to make real telemetry architecture decisions
- 5-6 correct: Good foundation; review the scenarios you missed
- Below 5: Re-read sections 01 and 02 before moving on

04.0 Conclusion

Content Outline:

- Course recap: what students accomplished
    - Understood why telemetry has become essential in modern software
    - Learned to make the three collection-layer decisions: where to process (client vs server), how to transport (serialization, format, reliability), and how often to send (throttling strategies)
    - Understood the two processing models — stream and batch — and how to choose between them based on latency requirements and cost
    - Connected all decisions into a coherent architecture: collection → transport → processing
- Connection to bigger picture
    - The next skills build directly on this: you will implement frontend telemetry collection (Skill 43) and build reports from collected data (Skill 44) — both assume the architectural understanding you now have
    - These decisions appear in every production system: observability tools, analytics pipelines, ML training infrastructure, and real-time fraud detection all apply exactly these concepts
- Next steps and continued learning
    - Explore: Apache Kafka documentation — the most widely used stream processing backbone

    - Explore: Apache Spark or dbt — leading batch processing tools
    - Read: "Designing Data-Intensive Applications" by Martin Kleppmann — chapters on stream and batch processing go deep on everything covered here
    - Practice: look at the telemetry architecture of an open-source project you use and identify which decisions they made
- Encouragement
    - Telemetry architecture is invisible when it works — and catastrophic when it doesn't. You now have the vocabulary and framework to design systems that work.

Note: This is conclusion only — no theory, exercises, or assessment.